CHAPTER 14: OBJECT ORIENTED DATA MODELING

Modern Database Management
11th Edition, International Edition

Jeffrey A. Hoffer, V. Ramesh, Heikki Topi

OBJECTIVES

- Definition of terms
- Describe phases of object-oriented development life cycle
- State advantages of object-oriented modeling
- Compare object-oriented model with E-R and EER models
- Model real-world application using UML class diagram
- Provide UML snapshot of a system state
- Recognize when to use generalization, aggregation, and composition
- Specify types of business rules in a class diagram

WHAT IS OBJECT-ORIENTED DATA MODELING?

- Centers around objects and classes
- Involves inheritance
- Encapsulates both data and behavior
- Benefits of Object-Oriented Modeling
  - Ability to tackle challenging problems
  - Improved communication between users, analysts, designer, and programmers
  - Increased consistency in analysis and design
  - System robustness
  - Reusability of analysis, design, and programming results

OO VS. EER DATA MODELING

Object Oriented	EER
Class	Entity type
Object	Entity instance
Association	Relationship
Inheritance of attributes	Inheritance of attributes
Inheritance of behavior	*No representation of behavior*

Object-oriented modeling is frequently accomplished using the **Unified Modeling Language (UML)**

OBJECT

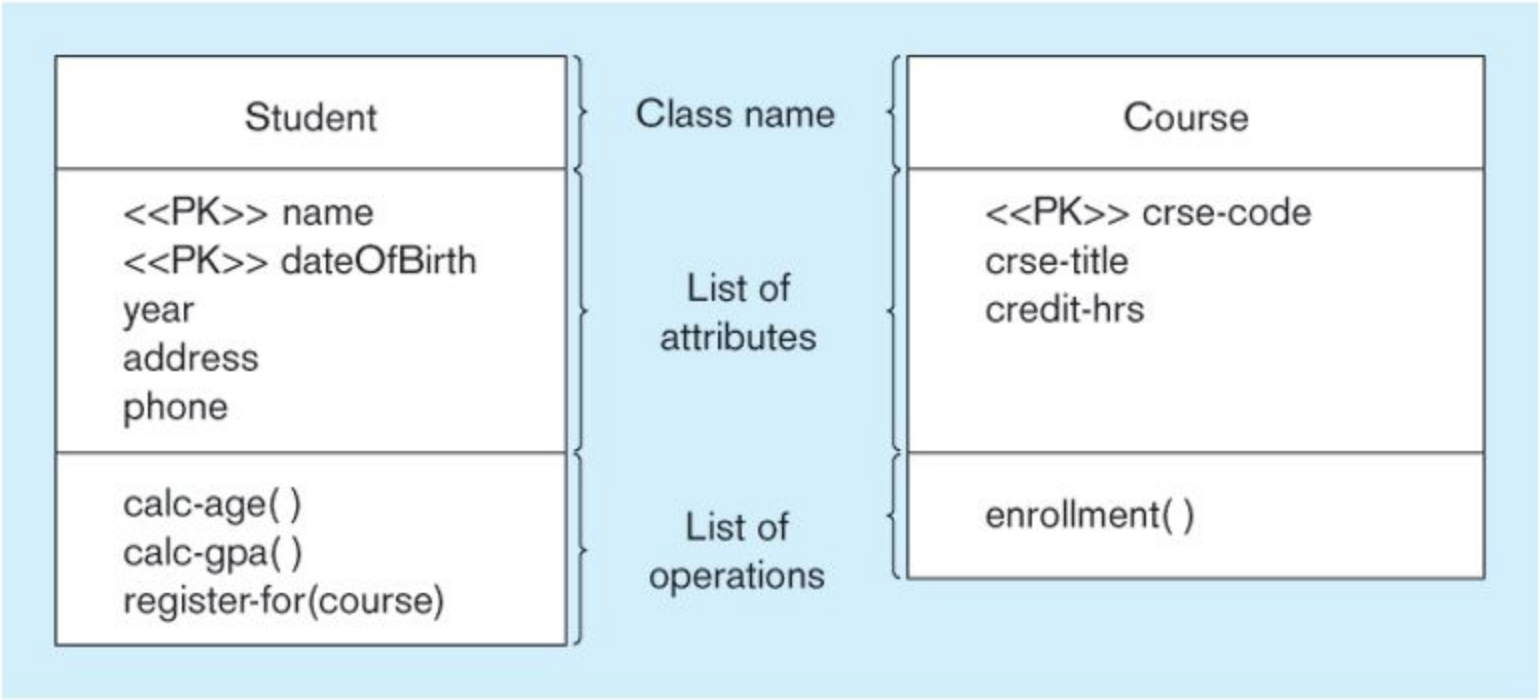
- An entity that has a well-defined role in the application domain, as well as state, behavior, and identity
  - Tangible: person, place or thing
  - Concept or Event: department, performance, registration
  - Artifact of the Design Process: user interface, controller, scheduler

Objects exhibit BEHAVIOR as well as attributes
 Different from **entities**

STATE, BEHAVIOR, IDENTITY

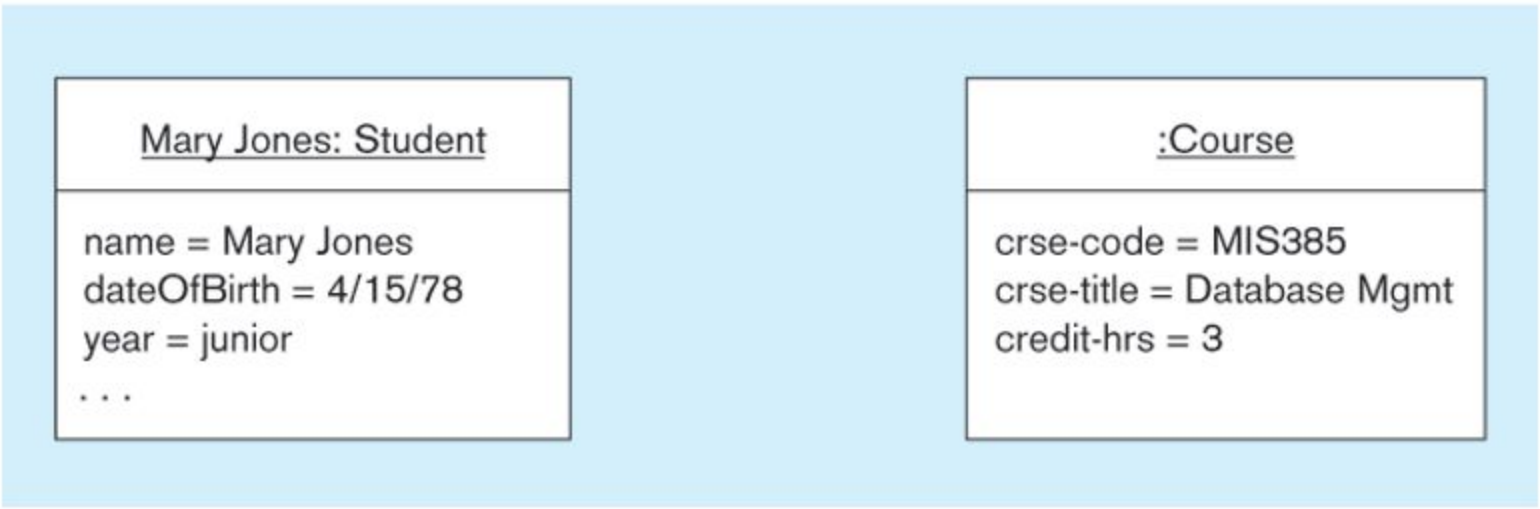
- State: attribute types and values
- Behavior: how an object acts and reacts
  - Behavior is expressed through operations that can be performed on it
- Identity: every object has a unique identity, even if all of its attribute values are the same

Figure 14-2a UML class and object diagrams - Class diagram showing two classes



Class diagram shows the static structure of an object-oriented model: object classes, internal structure, relationships.

Figure 14-2b UML class and object diagrams - Object diagram with two instances



Object diagram shows instances that are compatible with a given class diagram.

OPERATIONS

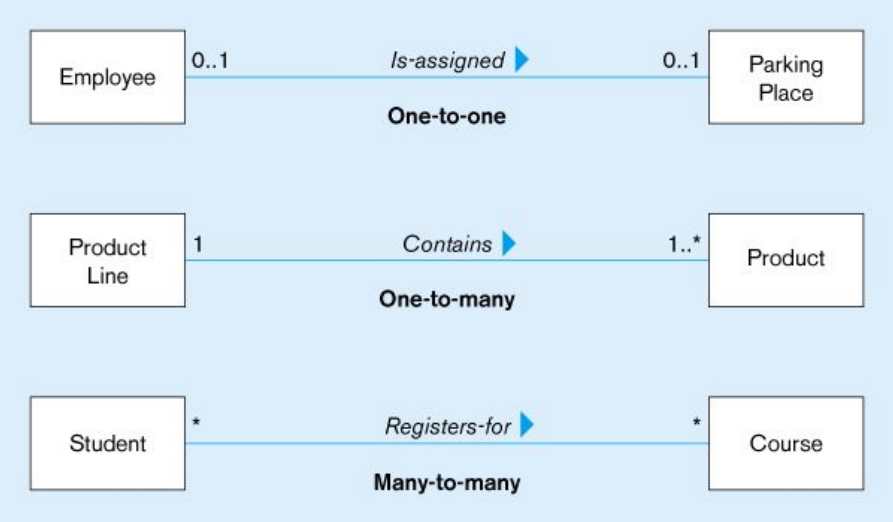
- **A function or service that is provided by all instances of a class**
- **Types of operations:**
  - **Constructor**: creates a new instance of a class
  - **Query**: accesses the state of an object but does not alter its state
  - **Update**: alters the state of an object
  - **Scope**: operation applying to the class instead of an instance

Operations implement the object's **behavior**

ASSOCIATIONS

- **Association**:
  - Relationship among object classes
- **Association Role**:
  - Role of an object in an association
  - The end of an association where it connects to a class
- **Multiplicity**:
  - How many objects participate in an association. Lower-bound..Upper bound (cardinality)

Figure 14-3:
Association relationships of different degrees



Lower-bound – upper-bound

Represented as:

0..1, 0..*, 1..1, 1..*

Similar to minimum/maximum cardinality rules in EER

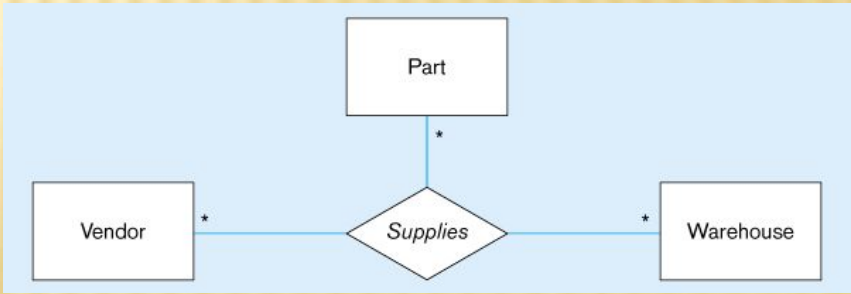
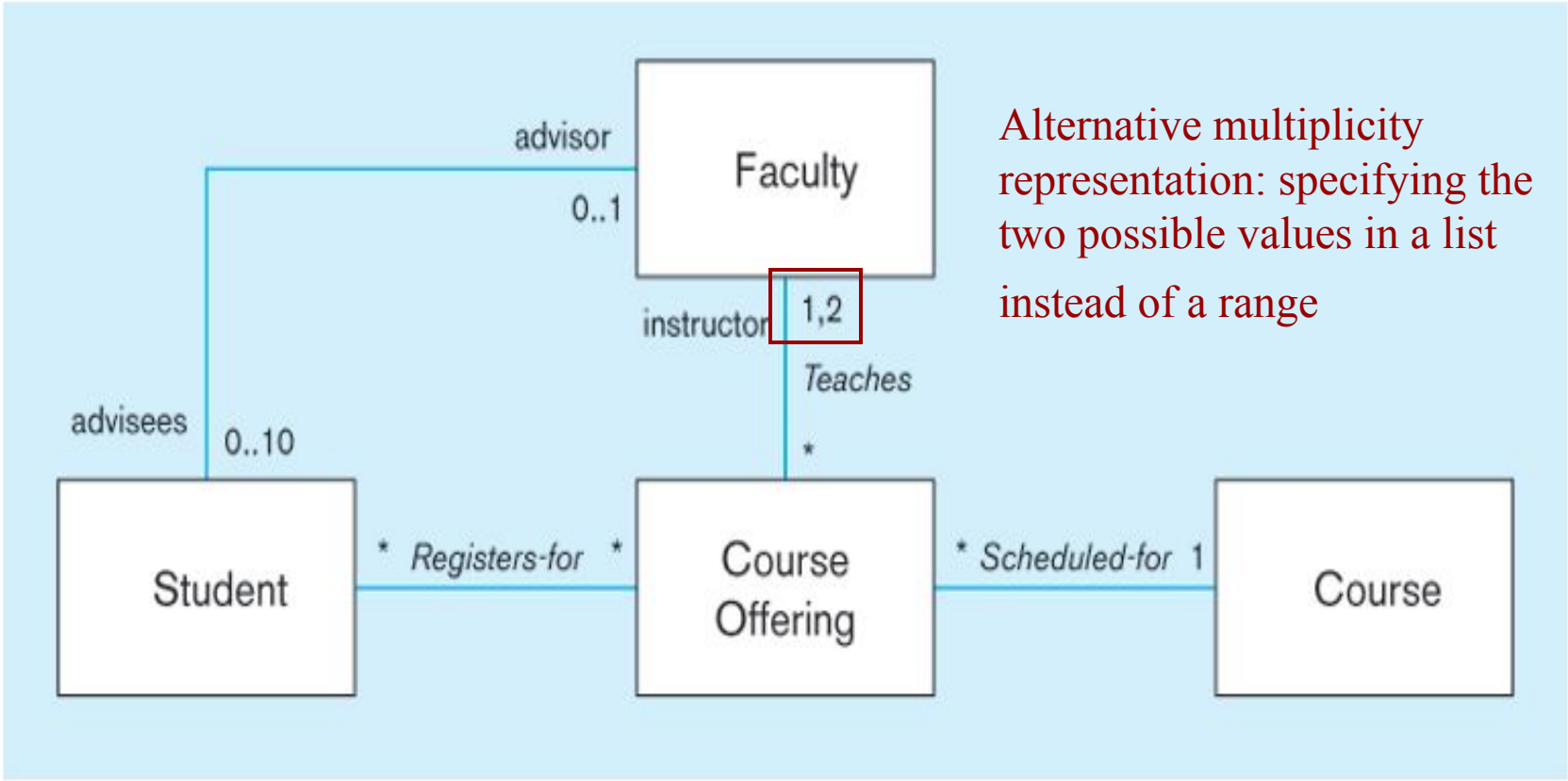


Figure 14-4a Examples of binary association relationships - University example



Alternative multiplicity representation: specifying the two possible values in a list instead of a range

Figure 14-4b Examples of binary association relationships - Customer order example

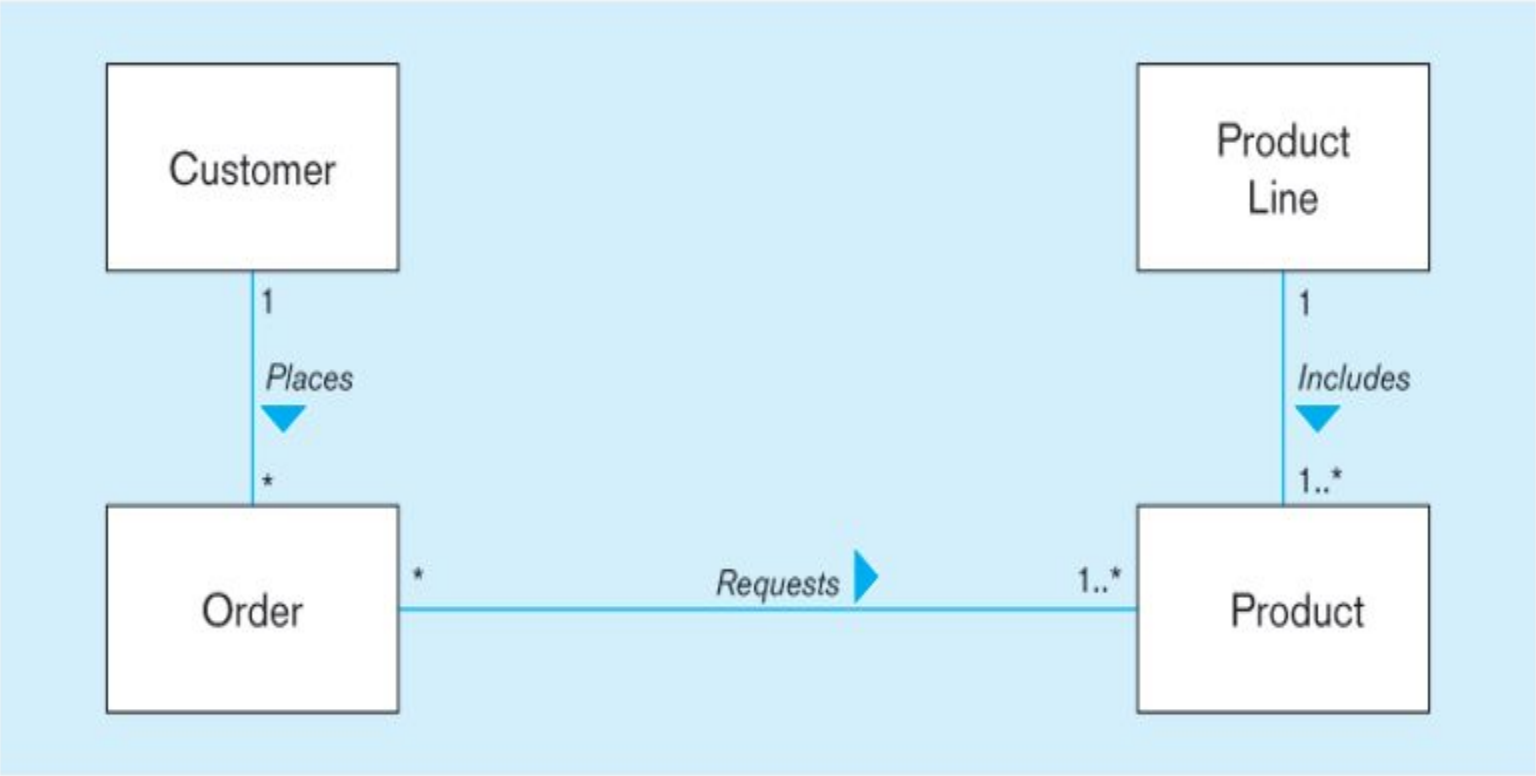
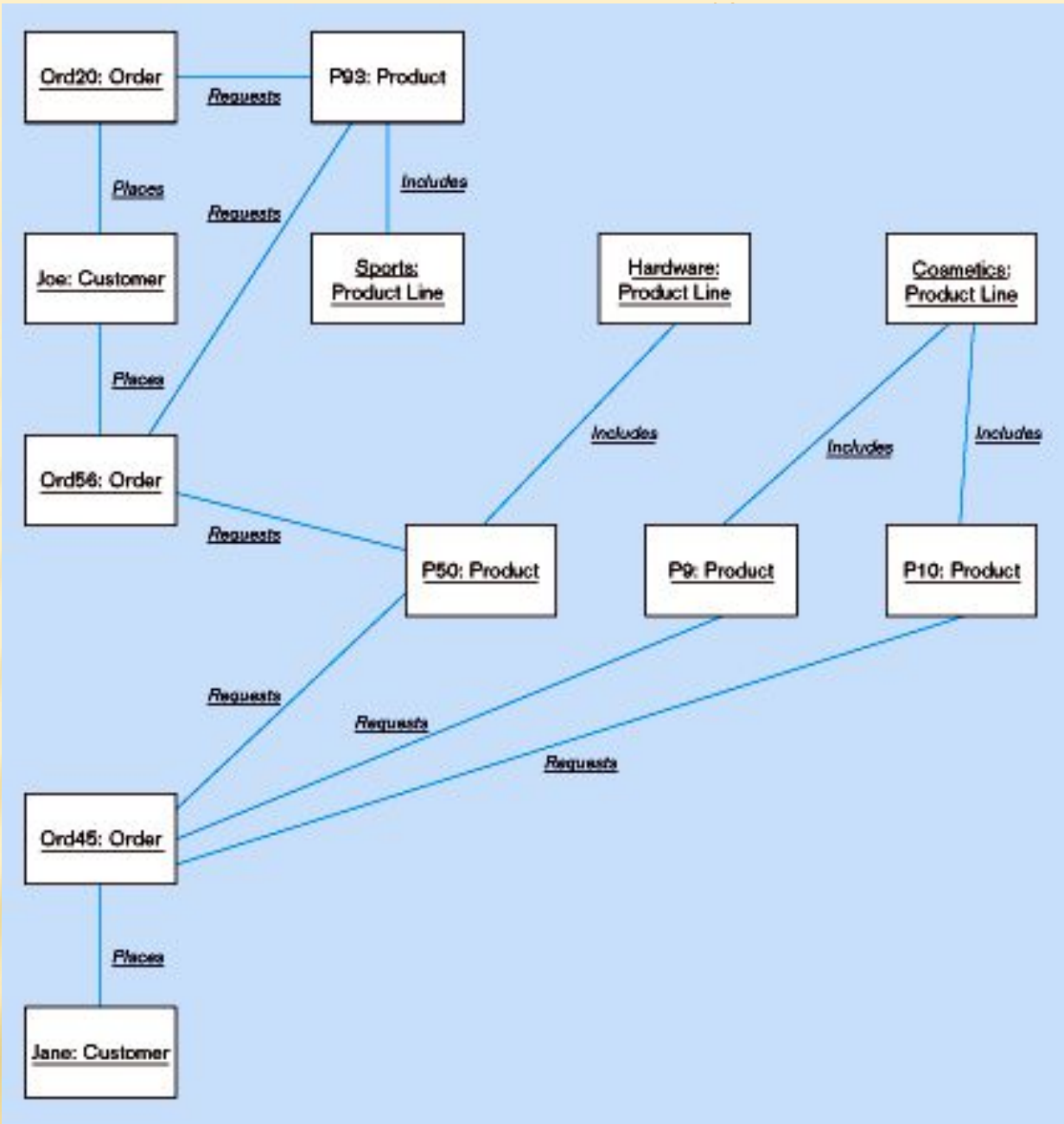


Figure 14-5: Object diagram for customer order example



ASSOCIATION CLASS

- An association that has attributes or operations of its own or that participates in relationships with other classes
- Like an associative entity in ER model

Figure 14-6a Association class and link object - Class diagram showing association classes

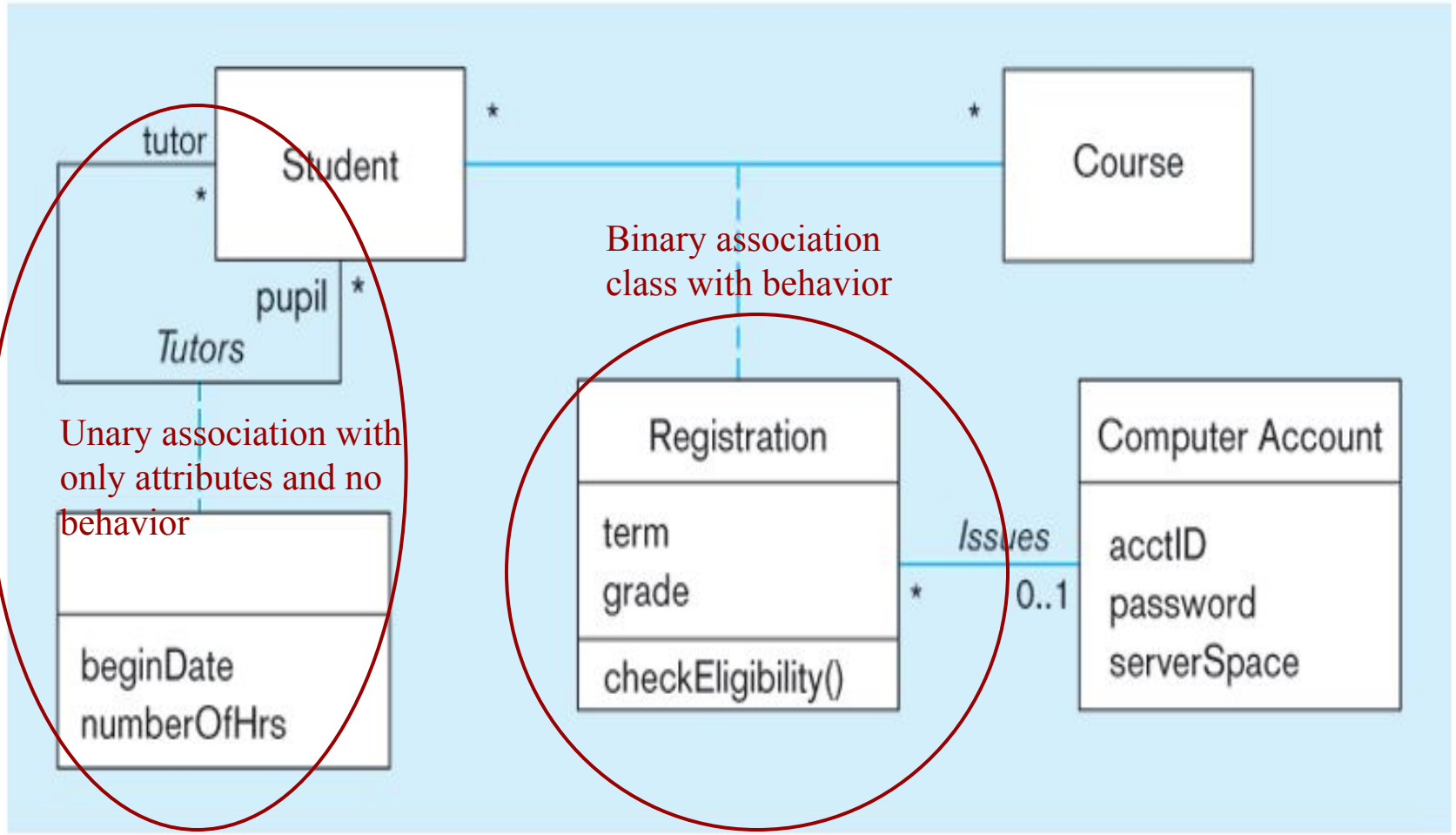


Figure 14-7: Ternary relationship with association class

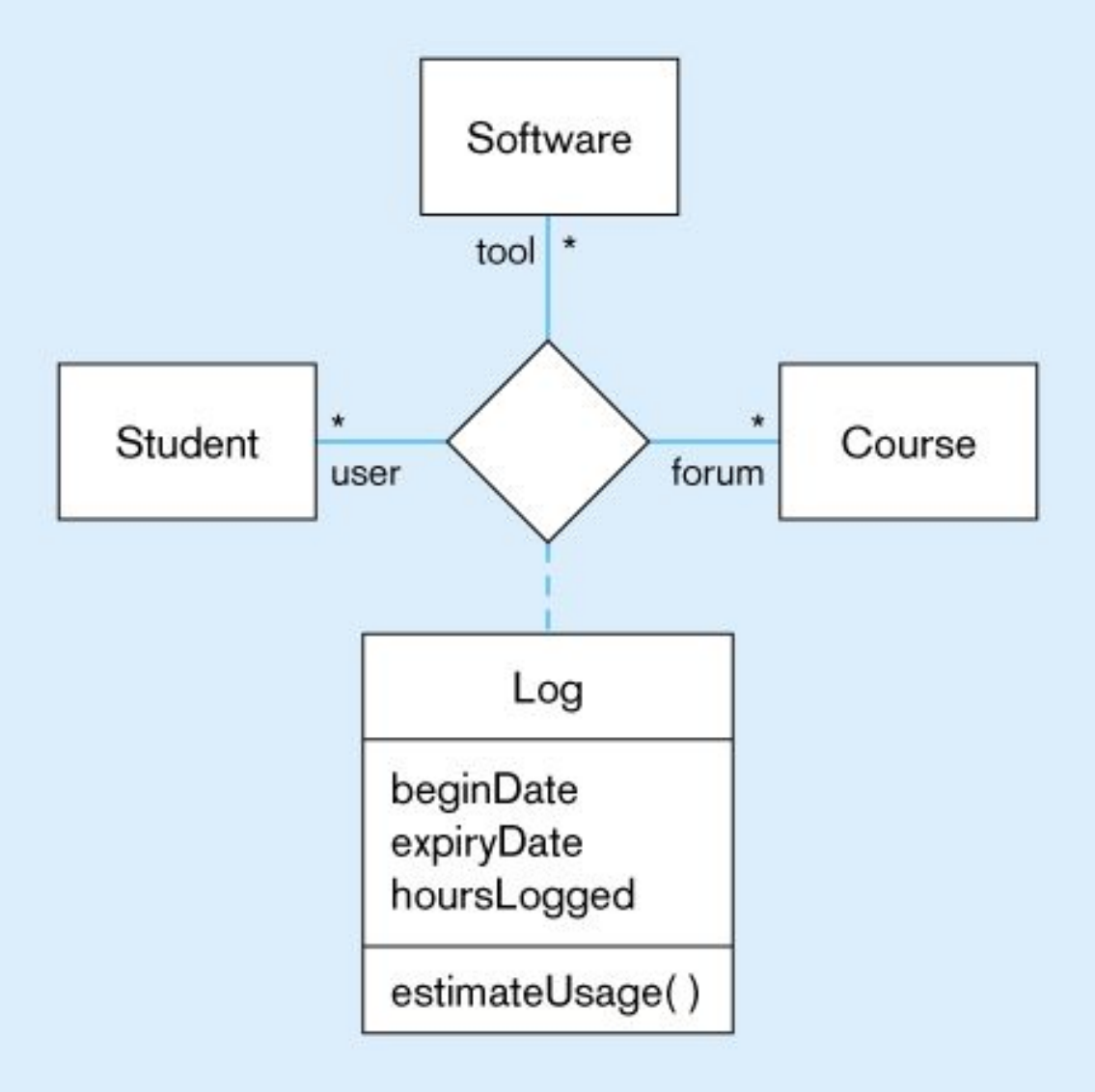
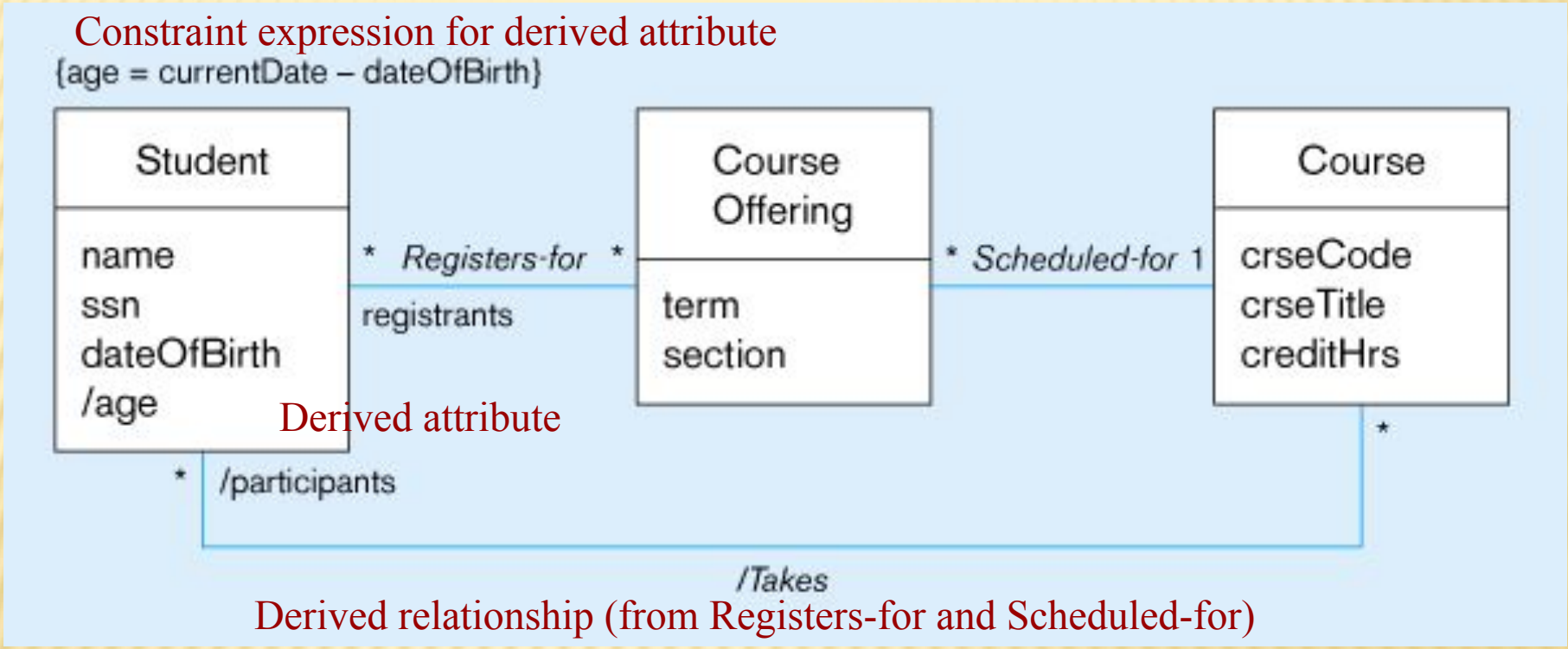


Figure 14-8: Derived attribute, association, and role



Derived attributes and relationships shown with / in front of the name

GENERALIZATION/SPECIALIZATION

- Subclass, superclass
  - similar to subtype/supertype in EER
- Common attributes, relationships, ***AND operations***
- Disjoint vs. Overlapping
- Complete (total specialization) vs. incomplete (partial specialization)
- Abstract Class: no direct instances possible, but subclasses may have direct instances
- Concrete Class: direct instances possible

Figure 14-9a: Examples of generalization, inheritance, and constraints

Employee superclass with three subclasses

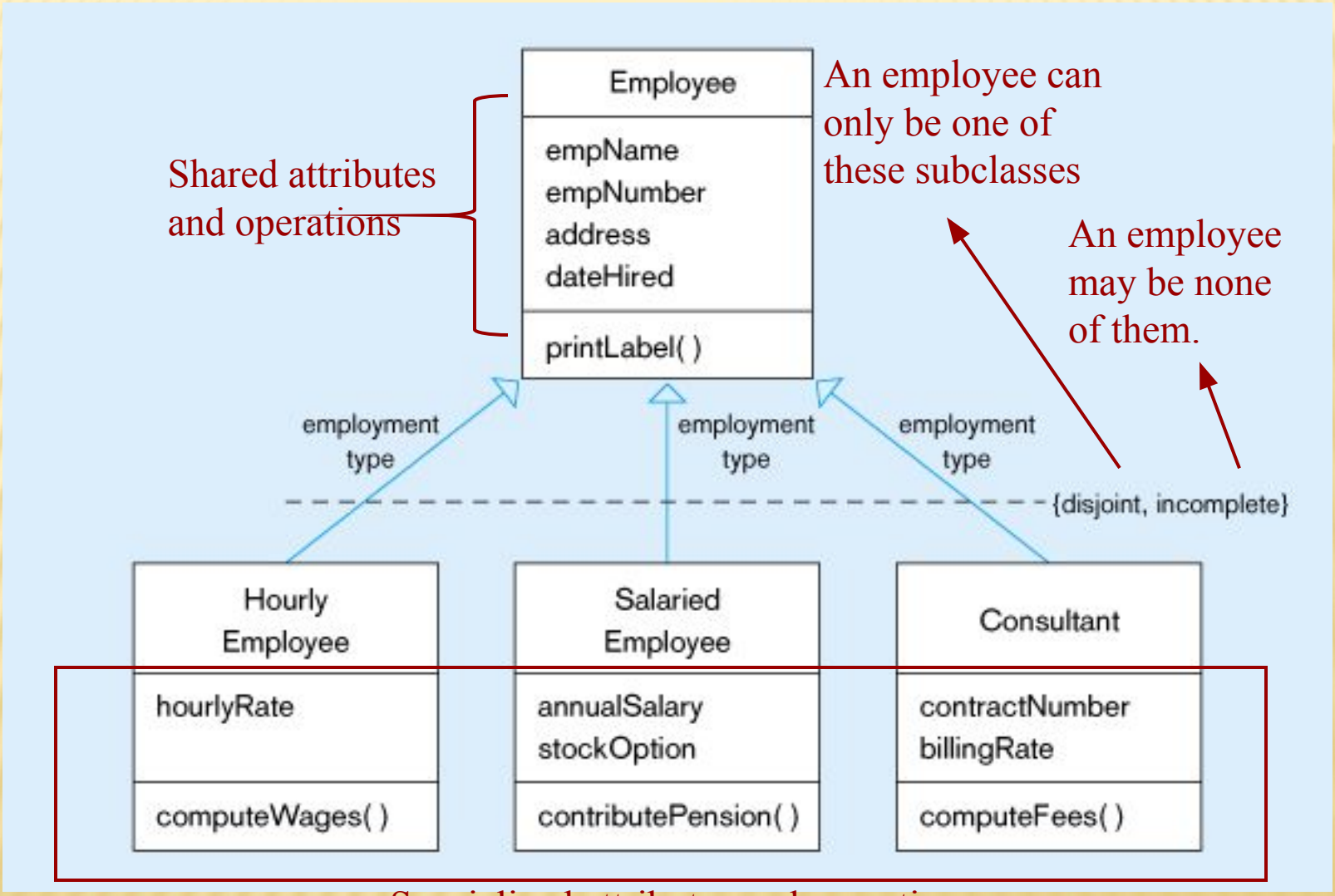
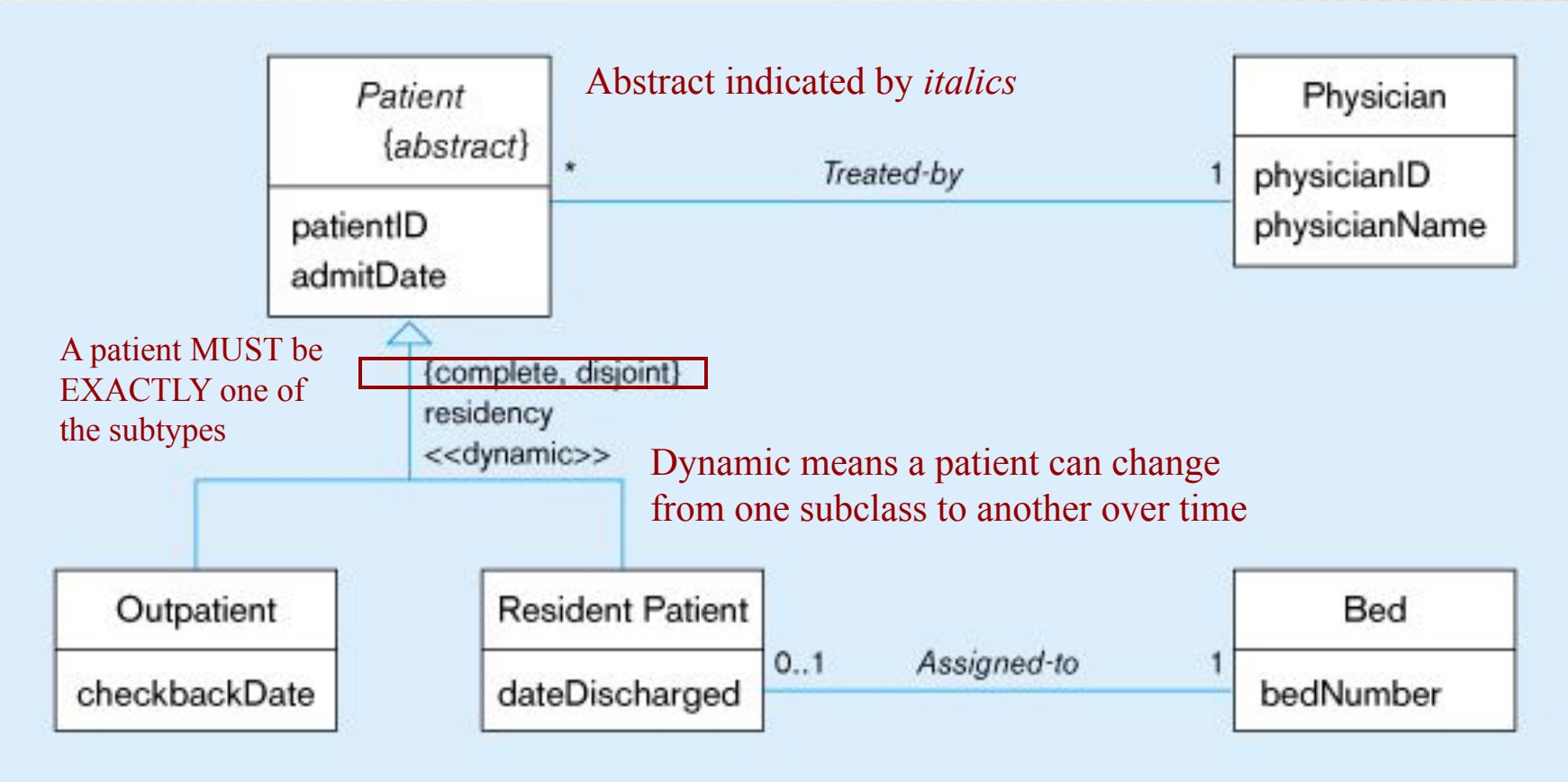


Figure 14-9b: Examples of generalization, inheritance, and constraints

Abstract patient class with two concrete subclasses



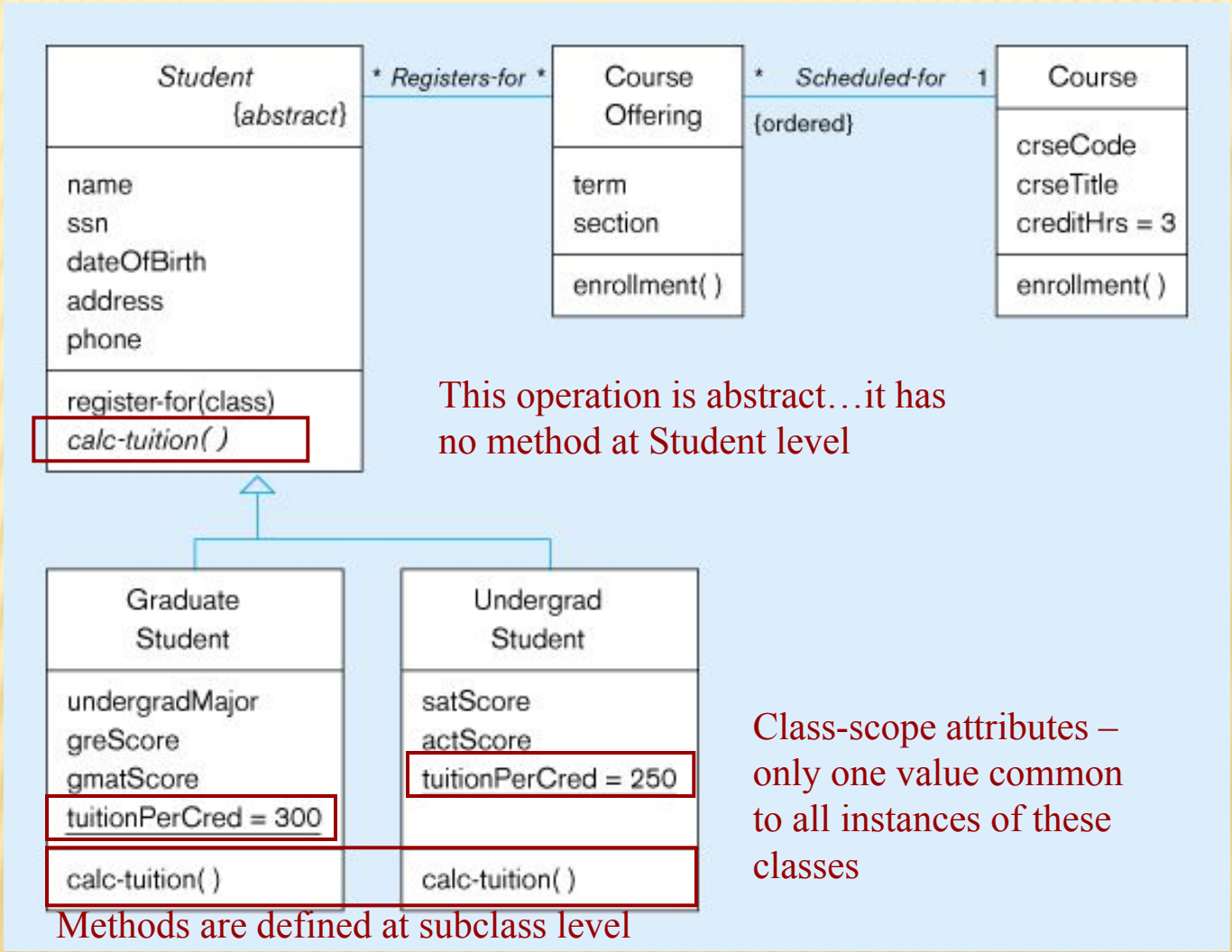
CLASS-SCOPE ATTRIBUTE

- Specifies a value common to an entire class, rather than a specific value for an instance.
- Represented by underlining
- “=“ is initial, default value.

POLYMORPHISM

- Abstract Operation: Defines the form or protocol of the operation, but not its implementation
- Method: The implementation of an operation
- *Polymorphism*: The same operation may apply to two or more different classes in different ways

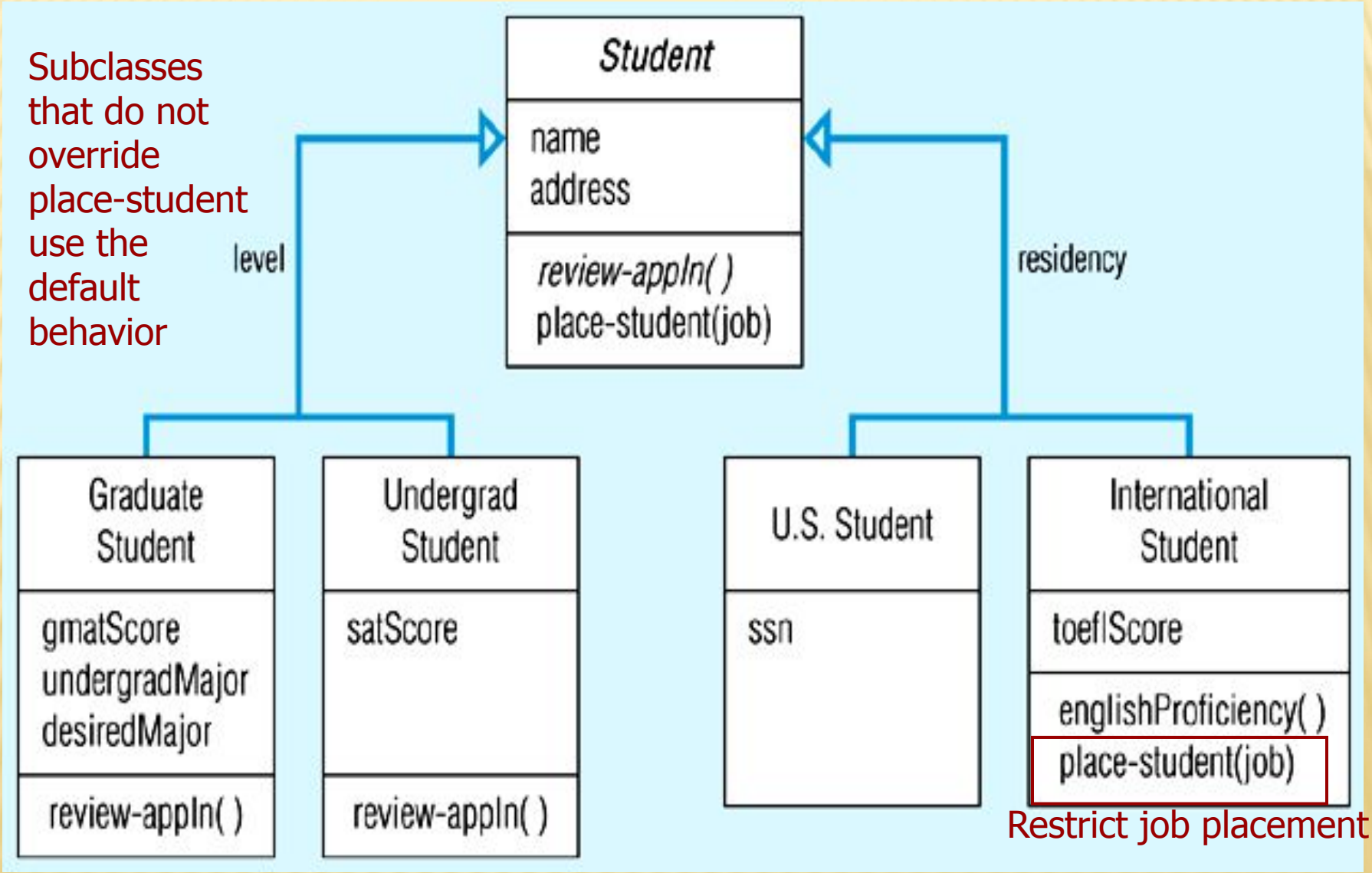
Figure 14-11: Polymorphism, abstract operation, class-scope attribute, and ordering



OVERRIDING INHERITANCE

- Overriding: The process of replacing a method inherited from a superclass by a more specific implementation of that method in a subclass
  - For Extension: add code
  - For Restriction: limit the method
  - For Optimization: improve code by exploiting restrictions imposed by the subclass

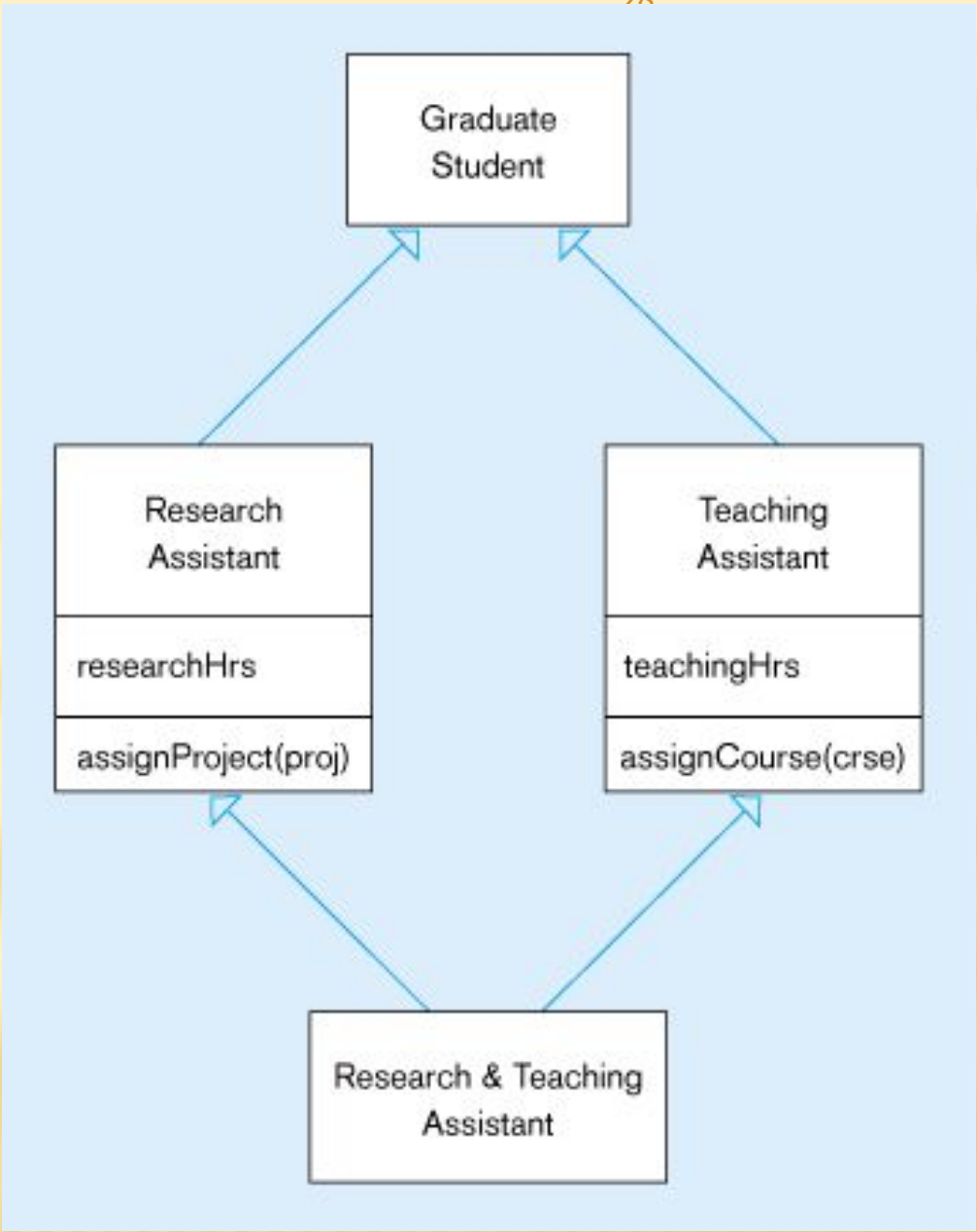
Figure 14-12: Overriding inheritance



MULTIPLE INHERITANCE

- Multiple Classification: An object is an instance of more than one class
- Multiple Inheritance: A class inherits features from more than one superclass

Figure 14-13
Multiple inheritance



BUSINESS RULES

- See Chapters 3 and 4
- Implicit and explicit constraints on objects – for example:
  - cardinality constraints on association roles
  - ordering constraints on association roles
- Business rules involving two graphical symbols:
  - labeled dashed arrow from one to the other
- Business rules involving three or more graphical symbols:
  - note with dashed lines to each symbol

Figure 14-17: Representing business rules

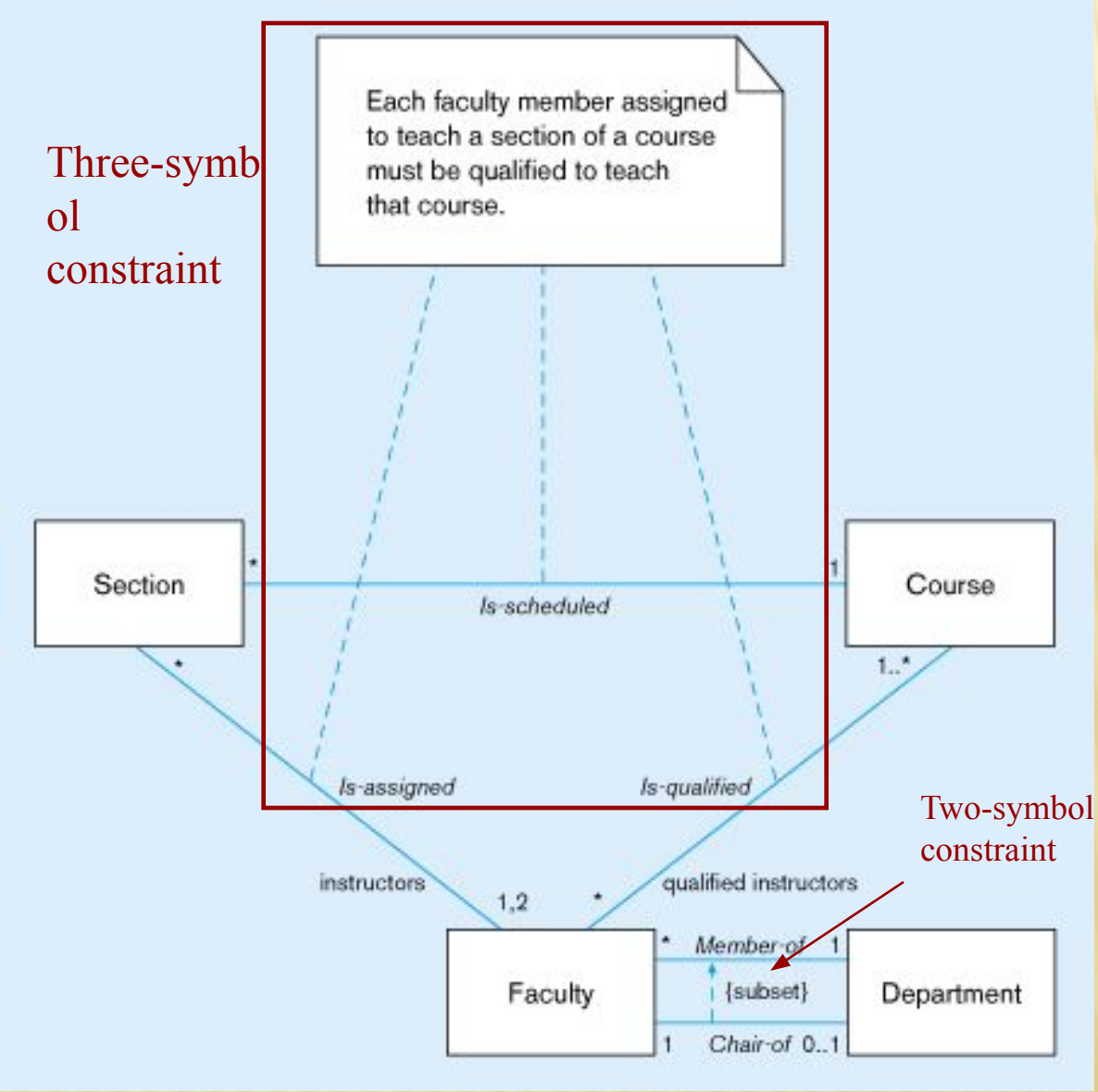


Figure 14-18 Class diagram for Pine Valley Furniture Company